

Installer deux extensions

- **Wappalyzer** : C'est un outil (souvent une extension de navigateur) qui **détecte et identifie les différentes technologies** (comme les CMS, frameworks, outils d'analyse) utilisées pour construire et faire fonctionner un site web que vous visitez.
- **React Developer Tools** : C'est une extension de navigateur **essentielle pour les développeurs React**. Elle permet d'inspecter, de déboguer et d'analyser les performances des applications web construites avec React, en donnant accès à la structure des composants, leurs états et propriétés.

1. Installer Node.js

Avant de commencer, vous devez installer **Node.js** car il est nécessaire pour gérer React et ses outils.

1. **Téléchargez Node.js** :
 - Rendez-vous sur nodejs.org et téléchargez le.
2. **Installez Node.js** :
 - Suivez les étapes d'installation (c'est simple : suivant, suivant, terminer).
3. **Vérifiez l'installation** :
 - Ouvrez votre terminal et tapez les commandes suivantes pour vérifier :

```
node -v
```

```
npm -v
```

2. Créer un nouveau projet React

1. **Naviguez vers le dossier où vous voulez créer le projet (par exemple f:\react)**
2. **Ouvrez le répertoire 'react' sur Visual studio**
3. **Ouvrez le terminal**
4. **Tapez la commande suivante pour générer automatiquement un projet React, remplacez mon-projet par le nom de votre projet :**

```
npx create-react-app monProjet
```

ou

```
npx create-react-app .
```

- **npx create-react-app mon-projet** : Utilisez cette commande lorsque vous souhaitez créer un nouveau dossier pour votre projet. C'est utile pour garder vos projets organisés et éviter de mélanger les fichiers de différents projets.
- **npx create-react-app .** : avec un point à la fin. Utilisez cette commande lorsque vous avez déjà créé un dossier pour votre projet et que vous souhaitez initialiser le projet React directement dans ce dossier. Assurez-vous que le dossier est vide ou ne contient pas de fichiers qui pourraient entrer en conflit avec les fichiers générés par Create React App. Excellent

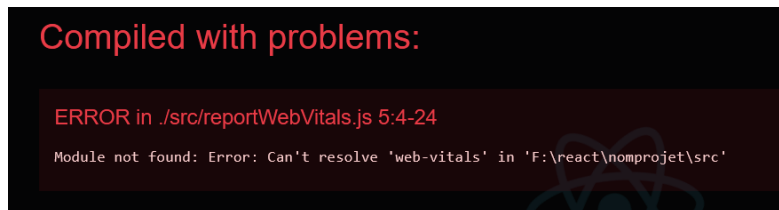
5. Entrez dans le dossier du projet :

```
cd mon-projet
```

6. Lancez l'application en développement :

```
npm start
```

Cela démarre un serveur local et ouvre votre application dans le navigateur à l'adresse : <http://localhost:3000>.

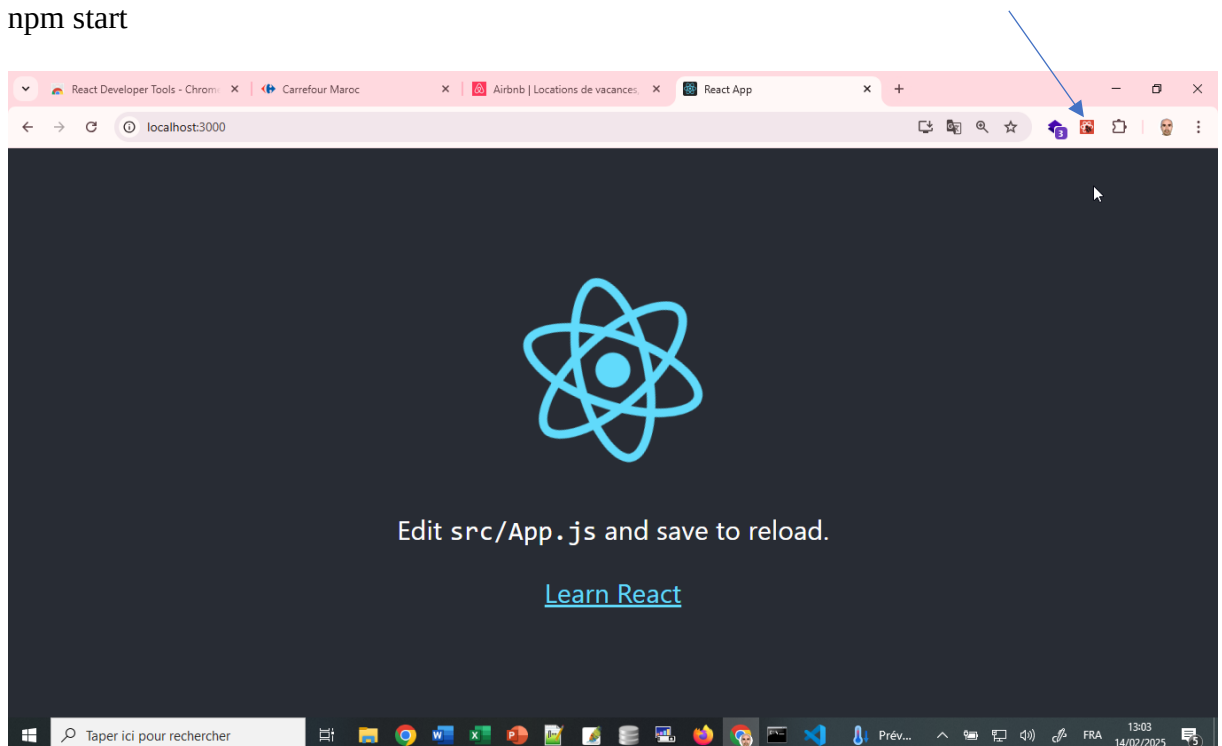


Si on a ce problème on tape :

```
npm install web-vitals
```

lancer

```
npm start
```



Ordre d'exécution complet :

1. Le navigateur charge `index.html`
2. `index.js` est exécuté en premier :
 - Monte l'application React dans `<div id="root">`
 - Rend le composant `<App />`

3. `App.js` est rendu et exécuté
4. Les composants enfants sont rendus récursivement

```
function App() {  
  return (  
    <div className="App">  
      <header className="App-header">  
        <img src={logo} className="App-logo" alt="logo" />  
        <p>  
          Edit <code>src/App.js</code> and save to reload.  
        </p>  
        <a  
          className="App-link"  
          href="https://reactjs.org" I  
          target="_blank"  
          rel="noopener noreferrer"  
        >  
          Learn React  
        </a>  
      </div>  
    )  
  )  
}
```

Ce que vous voyez ici n'est du html, mais c'est du JSX

Dans cette partie, on va voir du jsx

Jsx = combinaison de javascript+xml

Ce qui est entre `<DIV>` n'est pas du html, c'est le signe `<` et signe `>` et les lettres D, I et V

On peut mettre une balise ouvrante et fermante vide, on l'appelle un fragement:

`<>`

`</>`

Pour s'assurer on met :

```
<div style="color:red">
```

Et ça donnera une erreur

Pour le corriger, la méthode d'écrire du style avec jsx est :

```
<div style={{color:"red"}}>
```

La première accolade `{}` est ce qui dit à JSX : "Hé, ce que tu vois ici n'est pas une simple chaîne de caractères, c'est du code JavaScript que tu dois évaluer." Sans cette accolade, React interpréterait `color:"red"` comme une simple chaîne de caractères et non comme un objet JavaScript représentant des styles.

La Deuxième accolade à l'intérieur de l'expression JavaScript (délimitée par la première accolade), on utilise un *objet JavaScript* pour définir les styles. Cet objet est un ensemble de paires clé-valeur

Avec jsx :

const element=<h1>Bonjour tout le monde</h1>

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import App from './App';

const root = ReactDOM.createRoot(document.getElementById('root'));
const element=<h1> bonjour tout le monde</h1>
root.render(
  element
  //<App />
);
```

Remarquez qu'il n'y a pas de guillemets autour de la balise <h1>. C'est du JSX.

Ou directement la mise dans le render

```
const root = ReactDOM.createRoot(document.getElementById('root'));
root.render(
  <h1> bonjour tout le monde</h1>
  // <App />
);
```

Quand vous écrivez <h1>bonjour tout le monde</h1> sans guillemets dans un fichier React, le compilateur JSX (comme Babel) le transforme *avant* que le code ne soit exécuté par le navigateur. Il le transforme en quelque chose comme ceci (simplifié) :

React.createElement('h1', null, 'bonjour tout le monde');

React.createElement est une fonction de React qui crée un élément React. Le premier argument est le type d'élément (ici 'h1'), le deuxième est un objet de propriétés (ici null car il n'y en a pas), et le troisième est le contenu (ici 'bonjour tout le monde').

On va ajouter du javascript dans app.js

```
function App() {
  let langage="REACT"
  return (
    <div className="App">
      <header className="App-header">
        <img src={logo} className="App-logo" alt="logo" />
        <p>
          Edit <code>src/App.js</code> and save to reload.
        </p>
        <a
```

```

        className="App-link"
        href="https://reactjs.org"
        target="_blank"
        rel="noopener noreferrer"
      >
        Learn {langage}
      <br/>
      {2*5}
    </a>
  </header>
</div>
);
}

```

On peut mettre let langage="REACT" à l'extérieur de la fonction.

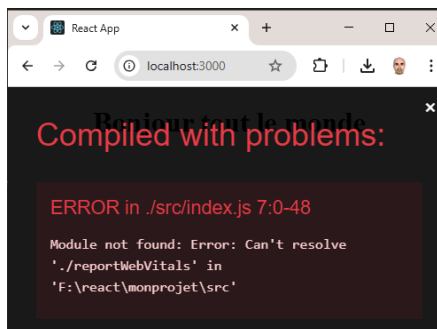
On supprime :

App.test.js , reportWebVitals.js, setupTests.js, les deux logos, favicon.icon

Manifest.json, Robot.txt, logo.svg, index.css, .gitignore, README.md

On garde : index.html, index.js,app.js,app.css,package-lock.json, package.json

Quand on va supprimer index.css il faut le supprimer de index.js `//import './index.css';`

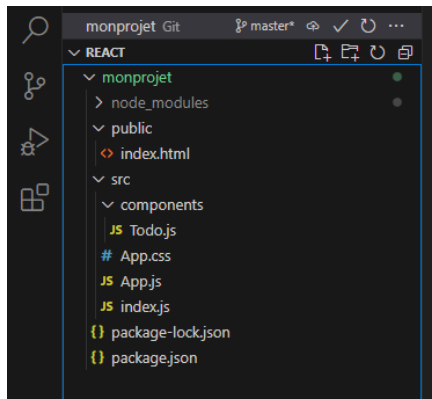


Avec cette erreur : supprimer dans index.js ces deux lignes :

```
import reportWebVitals from './reportWebVitals';
```

et reportWebVitals qui sert pour les analytics et analyse de performance

```
reportWebVitals();
```



On va vider le fichier App.css et mettre dedans

```
h1 {  
  text-align: center;  
  color :red  
}
```

Et on aura :

```
import React from 'react';  
import './App.css';  
function App() {  
  let langage="REACT"  
  return (  
    <div >  
      <h1> Learn {langage}    </h1>  
    </div>  
  );  
}  
export default App;
```

Un composant est une fonction ou une classe.

On va d'abord transformer les fonctions en classes dans le fichier App.js

```
import React, { Component } from 'react';  
import './App.css';  
class App extends Component{  
  render(){  
    let langage="REACT"  
    return (  
      <div >  
        <h1 > Learn {langage}</h1>  
      </div>  
    );  
  }  
}  
export default App;
```

En React, lorsque vous créez un composant sous forme de classe (en utilisant `class App extends Component`), vous devez définir une méthode nommée `render()`. C'est la méthode `render()` qui renvoie le JSX à afficher.

`render() { ... }` : Définit la méthode `render()`. C'est cette méthode qui est appelée par React pour déterminer ce qui doit être affiché à l'écran.

`return ( ... );` : La méthode `render()` *doit* renvoyer quelque chose, généralement du JSX. J'ai ajouté un `return` pour que la méthode renvoie l'élément JSX. Sans cela, la méthode retourne `undefined`, et React n'affiche rien.

Après on va créer notre premier composant

Aller sur scr

Puis créer un répertoire « **components** »

Puis créer un fichier : **Todo.js**

Je vais copier le code du **App.js** et le coller dans **Todo.js** et l'adapter

```
import { Component } from 'react';
class Todo extends Component {
  render () {
    return (
      <div >
        <h1>c est mon premier component to do</h1>
      </div>
    )
  }
}
export default Todo;
```

Ensuite je vais aller dans **App.js** et appeler **Todo**

```
import { Component } from 'react';
import Todo from './components/Todo';

class App extends Component {
  render () {
    return (
      <Todo />
    )
  }
}
export default App;
```

inner component

En JavaScript, un **inner component** fait généralement référence à un composant imbriqué à l'intérieur d'un autre composant.

Changer Todo.js par :

```
import React, { Component } from 'react';
import '../App.css';
class Todo extends Component {
  render () {
    return (
      <div>
        <h1>
          Introduction à React
        </h1>
        <p>
          React est une bibliothèque JavaScript populaire utilisée pour
          construire des interfaces utilisateur (UI). Développée par Facebook, elle
          permet de créer des applications web interactives et dynamiques de manière
          efficace.
          Avant de commencer avec React, il est utile de maîtriser certains
          concepts et technologies de base. Voici une liste des éléments clés à
          connaître :
          </p>
          <ul>
            <li> HTML </li>
            <li> CSS</li>
            <li> JavaScript </li>
          </ul>
        </div>
      );
    }
  }
  export default Todo;
```

On va créer des composants à l'intérieur du component Todo.

On va commencer par créer un fichier .js pour chaque élément dans Todo.js

Il y a Titre.js

```
import { Component } from 'react';
class Titre extends Component {
  render () {
    return (
      <h1>
        Introduction à React
      </h1>
    );
  }
}
```



```

    )
  }
}
export default Titre;

```

Il y a Corps.js

```

import { Component } from 'react';
class Corps extends Component {
  render () {
    return (
      <p>
        React est une bibliothèque JavaScript populaire utilisée pour
        construire des interfaces utilisateur (UI). Développée par Facebook, elle
        permet de créer des applications web interactives et dynamiques de manière
        efficace.

        Avant de commencer avec React, il est utile de maîtriser certains
        concepts et technologies de base. Voici une liste des éléments clés à
        connaître :
      </p>
    )
  }
}
export default Corps;

```

Et il y a Prerequis.js

```

import { Component } from 'react';
class Prerequis extends Component {
  render () {
    return (
      <ul>
        <li> HTML </li>
        <li> CSS</li>
        <li> JavaScript </li>

      </ul>
    )
  }
}
export default Prerequis;

```

Et on va faire appel aux composants de cette façon :

```

import { Component } from 'react';
import Titre from './Titre';
import Text from './Text';
import Prerequis from './Prerequis';

```

```

class Todo extends Component {
  render () {
    return (

      <Titre/>
      <Text/>
      <Prerequis/>
    </div>
  );
}
}
export default Todo;

```

Application du CSS

On a trois types :

Inline CSS

Vous pouvez ajouter du CSS directement dans les balises HTML en utilisant l'attribut `style`. C'est pratique pour des styles rapides et spécifiques, mais cela peut rendre le code HTML difficile à lire et à maintenir.

```

import React, { Component } from 'react';
import '../App.css';
class Titre extends Component{
  render(){
    return (
      <h1 style={{color:"red"}}>
        Introduction à React
      </h1>
    );
  }
}
export default Titre;

```

Internal CSS

```

import React, { Component } from 'react';
import '../App.css';
class Titre extends Component{
  render(){
    const style={color:"blue",fontSize:"40px"}
    return (
      <h1 style={style}>
        Introduction à React
      </h1>
    );
  }
}

```

```
}  
export default Titre;
```

External CSS

Vous pouvez lier un fichier CSS externe à votre document HTML en utilisant la balise `<link>`. C'est la méthode la plus courante et la plus recommandée pour les grands projets, car elle permet de séparer le contenu HTML et les styles CSS, rendant le code plus propre et plus facile à maintenir.

On va remettre le style avec App.css

```
import React, { Component } from 'react';  
import '../App.css';  
class Titre extends Component{  
  render(){  
    return (  
      <h1 >  
        Introduction à React  
      </h1>  
    );  
  }  
}  
export default Titre;
```

On peut aussi renommer App.css en style.css et l'importer dans index.js et supprimer import app.css de partout.

Comment ça se fait que dans ce code le style de h1 est pris d'un autre fichier app.css qui ne figure pas de lien dans ce fichier ?

Par défaut avec create-react-app, les fichiers CSS ont une portée globale. Si vous avez défini des styles pour les balises `<h1>` dans App.css (ou tout autre fichier CSS que vous importez globalement), ces styles affecteront tous les `<h1>` de votre application, même ceux dans le composant Todo.

Dans votre composant Todo, vous n'avez pas explicitement importé App.css. Cependant, si vous avez importé App.css dans un autre composant, comme App.js (le composant principal de votre application), et que App.css contient des règles CSS pour les balises `<h1>`, ces règles vont affecter votre `<h1>` dans le composant Todo en raison de la portée globale.

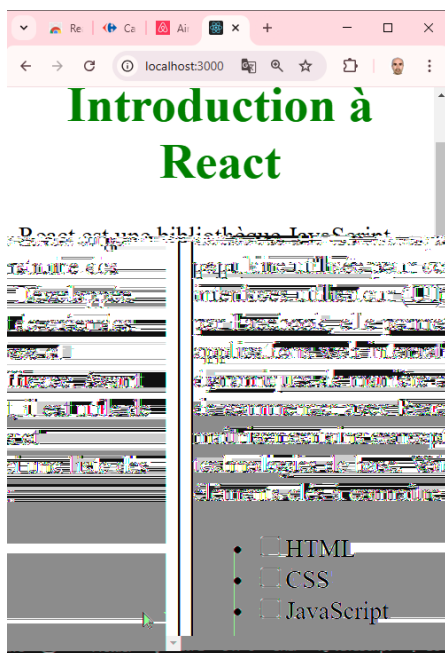
Dans cette partie nous allons tout remettre en fonction car React nous dit de travailler avec les fonctions au lieu des classes.

```
import React from 'react';
import './App.css';
import Todo from './components/Todo';
function App (){

  return (
    <div >
      <Todo/>
    </div>
  );
}
export default App;
```

Et on fait la même chose pour les autres composants : Todo, Titre, Text et Prerequis

Mettre les prérequis sous forme de checkbox



```
import React from 'react';
import './App.css';
function Prerequis (){
  return (
    <ul>
      <li>
        <input type="checkbox" id ="html" />
        <label for= "html" >HTML </label>
      </li>
      <li>
        <input type="checkbox" id ="css" />
        <label for= "css" >CSS </label>
      </li>
    </ul>
  );
}
```

```

        </li>
        <li>
          <input type="checkbox" id ="javascript" />
          <label for= "javascript" >JavaScript </label>
        </li>
      </ul>
    );
  }
export default Prerequis;

```

`<input type="checkbox" />` HTML. Notez l'espace avant le `/` qui est optionnel mais recommandé pour la compatibilité avec certains outils.

Les balises `<input>` sont des balises dites *orphelines* ou *vides* (void tags). Elles n'ont pas de balise de fermeture.

Étiquettes (Labels) : Pour l'accessibilité, il est important d'associer les cases à cocher à des étiquettes (`<label>`). Cela facilite la tâche des utilisateurs qui utilisent un lecteur d'écran ou qui ont des difficultés à cliquer sur de petites zones.

Javascript à l'intérieur de JSX (JavaScript XML)

Un autre exemple :

```

import React from 'react';
import '../App.css';
function Titre () {
  const profil = "programmation";
  let titre="";
  if (profil === "programmation")
  {
    titre="React"
  }
  else
  {
    titre="Reseau"
  }
  return (
    <h1 >
      Introduction à {titre}
    </h1>
  );
}
export default Titre;

```

Une autre façon :

```
import React from 'react';
import '../App.css';
function Titre (){
  const theme = "programmation";

  return (
    <h1 >
      Introduction à {(theme === "programmation")?"React":"Reseau"}
    </h1>
  );
}
export default Titre;
```

Le code suivant ne fonctionnera pas, pourquoi ? :

```
import React from 'react';
import '../App.css';
function Titre (){

  let titre="";

  return (
    <h1 >
      if (theme === "programmation")
      {
        titre="React"
      }
      else
      {
        titre="Reseau"
      }
      Introduction à {titre}
    </h1>
  );
}
export default Titre;
```

Parce que :

- Le return dans un composant React doit retourner une seule expression JSX. Vous ne pouvez pas insérer directement des instructions de contrôle de flux (comme if/else) dans le JSX. Vous devez d'abord évaluer la condition et stocker le résultat dans une

variable, ou utiliser un opérateur ternaire, puis utiliser cette variable ou cette expression ternaire à l'intérieur du JSX.

- Le bloc `return` d'un composant React doit renvoyer une seule expression JSX. Cette expression peut être une balise HTML unique, un fragment (`<> </>`), ou un appel à un autre composant.
- La structure `if/else` est une instruction de contrôle de flux en JavaScript, et non une expression. En JavaScript, une *expression* renvoie une valeur, tandis qu'une *instruction* exécute une action. Le JSX exige une expression.
- L'opérateur ternaire, que nous avons utilisé dans l'exemple précédent, est une *expression*, c'est pourquoi il fonctionne directement dans le JSX.

Rappel des règles fondamentales :

1. **Le `return` d'un composant React doit renvoyer *une seule* expression JSX (ou un fragment).**
2. **Vous ne pouvez pas directement imbriquer des instructions JavaScript (comme `if/else`, `for`, `while`) à l'intérieur du JSX.**
3. **Vous devez utiliser des *expressions* JavaScript (comme des opérateurs ternaires, des appels de fonctions, des variables) à l'intérieur du JSX.**

La clé est de se rappeler que le JSX est une syntaxe spéciale pour *décrire l'interface utilisateur*, et il a des règles différentes de JavaScript normal. Vous devez effectuer la logique JavaScript *avant* de générer le JSX.